Name:	Krish Sodhi, Marvin Sukumar, Riya Suryawanshi, Anushka Suvarna
UID:	2023300233, 2023300238, 2023300240, 2023300241
Subject:	Distributed Computing
Experiment No.:	4
Project title:	Travel Itinerary Planner
Problem Statement:	To implement clock synchronization using Lamport's Logical Clock Algorithm.
Objectives:	The objective of Lamport's logical clock algorithm in the Travel Itinerary Planner is to provide a consistent and ordered timestamping mechanism for user actions such as login, searching restaurants, booking transport, events, and adding attractions. This ensures that all operations across different distributed servers are executed in the correct sequence, maintaining logical consistency even if the physical clocks of the servers are not synchronized.
Theory:	**What are Lamport Clocks?** Lamport clocks represent time logically in a distributed system. They are also known as logical clocks. Instead of depending on physical time, Lamport clocks capture the "happens-before" relationship between events. This means they only care about the order in which actions happen, not the exact real-world time. **Why use Lamport clocks?** Time synchronization is a key problem in distributed systems. In a Travel Itinerary Planner, different servers handle different tasks (restaurant search, transport booking, event booking, adding attractions). If these servers rely on physical clocks, there may be clock skew (differences in their timings). This can cause incorrect ordering, like booking a restaurant before login. To solve this, Leslie Lamport proposed logical clocks, which focus only on event ordering through the "happens-before" relationship.

	Use in Travel Itinerary Planner In the context of our Travel Itinerary Planner, users perform actions such as: ● Searching restaurants ● Booking events ● Adding attractions Lamport's logical clocks assign a logical timestamp to each action. These timestamps ensure that actions are consistently ordered across distributed servers. For example: ● Searches restaurants at Lamport time 1, ● Books event at Lamport time 2, ● And adds attractions at Lamport time 3 The system guarantees that these steps will always be recognized in the correct order, even if different physical clocks on different servers are not synchronized.

Code:	**Server:**
	
	import socket
	import threading
	import sqlite3
	import json
	import time
	
	class LamportClock:
	  def __init__(self):
	  self.time=0
	
	  def update(self,received_time=0):
	  old_time=self.time
	  self.time=max(self.time,received_time)+1
	  print(f"[Server] Lamport Clock updated: {old_time} -> {self.time}")
	  return self.time
	
	clock=LamportClock()

```
def handle_client(conn,addr):
    global clock
    print(f"[Server] Connected to {addr}")
    print(f"[Server] Active Threads: {threading.active_count()-1}")

    try:
        data=conn.recv(1024).decode()

        msg=json.loads(data)

        action=msg["action"]
        params=msg["params"]
        ts=int(msg["lamport_time"])

        clock.update(ts)

        response=""
        conn_db=sqlite3.connect("travel.db")
        c=conn_db.cursor()

        if action=="RESTAURANTS":

            c.execute("SELECT name, price_range FROM restaurants WHERE
location=?",(params,))
            rows=c.fetchall()
            response=f"Restaurants in {params} with their price range: {rows}"

        elif action=="EVENT":

            c.execute("SELECT * FROM events WHERE name=?",(params,))
            row=c.fetchone()
            response=f"Event Booking Confirmed: {row}"

        elif action=="ATTRACTION":

            c.execute("SELECT * FROM attractions WHERE
location=?",(params,))
            row=c.fetchone()
            response=f"Attraction Added: {row}"

        else:
            response="Sorry! Invalid request"

        conn_db.close()
        time.sleep(50)
```

```
        response_with_time=f"{response} | Server
LamportTime={clock.time}"
        conn.send(response_with_time.encode())

    except Exception as e:
        print("Error:", e)


    conn.close()
    print(f"Disconnected with {addr}")

def start_server():
    server=socket.socket(socket.AF_INET,socket.SOCK_STREAM)
    server.bind(("localhost",5001))
    server.listen()
    print("Server is listening on port 5001...")

    while True:
        conn,addr=server.accept()
        thread=threading.Thread(target=handle_client,args=(conn,addr))
        thread.start()



start_server()
```

Client:

```
import socket
import json

class LamportClock:
    def __init__(self):
        self.time=0

    def tick(self):
        old_time=self.time
        self.time+=1
        print(f"[Client] Lamport Clock updated: {old_time} -> {self.time}")
        return self.time


clock=LamportClock()
```

```
def send_request(action,params):
    client=socket.socket(socket.AF_INET,socket.SOCK_STREAM)
    client.connect(("localhost",5001))

    lamport_time=clock.tick()
    msg={
      "action":action,
      "params":params,
      "lamport_time":lamport_time

    }
    client.send(json.dumps(msg).encode())

    response=client.recv(1024).decode()


    print("[Server] Response:",response)
    client.close()


ch=1
print("\nWelcome to PlanMyTrip!")
while(ch!=4):

    print('''

       Menu
       1. Search Restaurants
       2. Book Event
       3. Add Attraction
       4. Exit

    '''
    )

    ch=int(input("Enter your choice: "))

    if(ch==1):
        location=input("Enter location: ")
        send_request("RESTAURANTS",location)


    elif(ch==2):

        event_name=input("Enter event name: ")
```

```
        send_request("EVENT",event_name)


    elif(ch==3):
        loc=input("Enter attraction location: ")
        send_request("ATTRACTION",loc)

    else:

        print("Thank You! Bye")
```

Output:

Server:

```
C:\Users\SUVARNA HOUSE\Downloads\DC\Exp 4 codes>python server.py
Server is listening on port 5001...
[Server] Connected to ('127.0.0.1', 57371)
[Server] Active Threads: 1
[Server] Lamport Clock updated: 0 -> 2
[Server] Connected to ('127.0.0.1', 53460)
[Server] Active Threads: 2
[Server] Lamport Clock updated: 2 -> 3
[Server] Connected to ('127.0.0.1', 60258)
[Server] Active Threads: 3
[Server] Lamport Clock updated: 3 -> 4
Disconnected with ('127.0.0.1', 57371)
Disconnected with ('127.0.0.1', 53460)
Disconnected with ('127.0.0.1', 60258)
```

Client 1:

```
C:\Users\SUVARNA HOUSE\Downloads\DC\Exp 4 codes>python client.py

Welcome to PlanMyTrip!

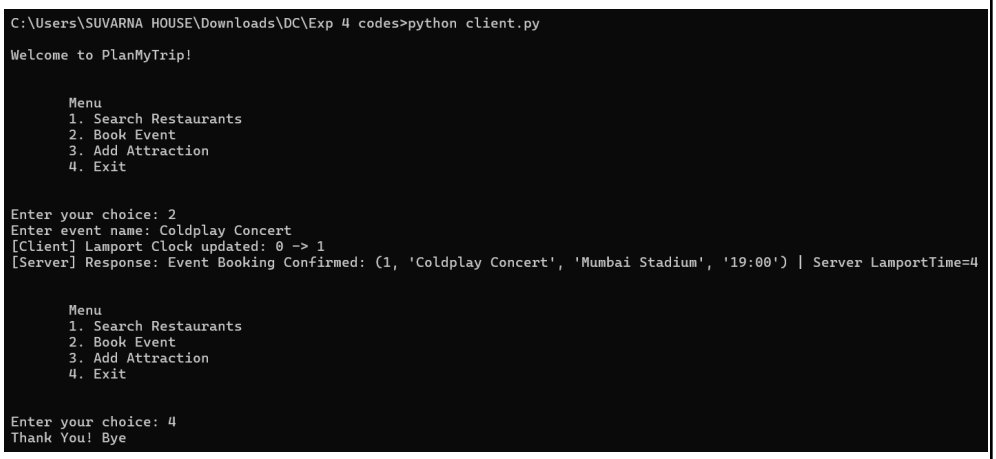
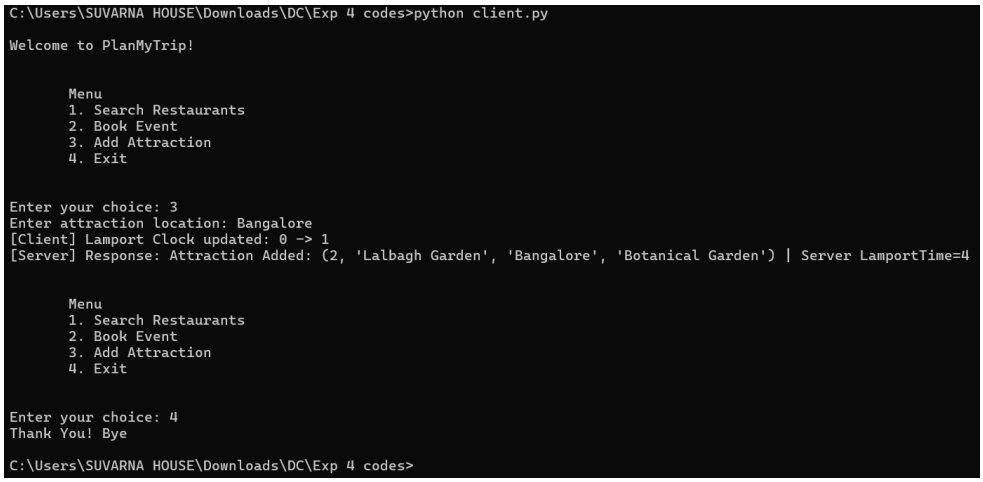

      Menu
      1. Search Restaurants
      2. Book Event
      3. Add Attraction
      4. Exit


Enter your choice: 1
Enter location: Mumbai
[Client] Lamport Clock updated: 0 -> 1
[Server] Response: Restaurants in Mumbai with their price range: [('Taj Hotel', 'High')] | Server LamportTime=4


      Menu
      1. Search Restaurants
      2. Book Event
      3. Add Attraction
      4. Exit


Enter your choice: 4
Thank You! Bye

C:\Users\SUVARNA HOUSE\Downloads\DC\Exp 4 codes>
```

	Client 2:  **Client 3:** 
Conclusion:	This experiment showed how Lamport's logical clock brings consistency to a distributed system by maintaining the correct order of events across different servers. In the travel itinerary planner, it ensured that actions like login, searching restaurants, booking transport, booking events, and adding attractions were always executed in the proper sequence. This provides users with a reliable and logically consistent experience, even if the physical clocks of the servers are not synchronized.